

Content-Based Image Retrieval

CS 5330, Computer Vision Project 2

Ashish Dasu Northeastern University

February 12, 2026

Project Overview

Content-Based Image Retrieval (CBIR) is the problem of ranking images in a database by visual similarity to a query image, using only pixel level content rather than metadata. The core idea is to map each image to a numeric feature vector and then sort the database by distance between the query's vector and every stored vector. Smaller distance means more similar.

This project implements a progression of feature representations applied to the provided photo collection (1,106 images). The simplest method compares raw center patch pixels using sum squared differences. From there, 2D and 3D color histograms capture global palette information, and splitting the image spatially before histogramming adds layout sensitivity. A combined color and texture feature incorporates Sobel gradient magnitude to distinguish smooth from textured scenes. Finally, 512 dimensional ResNet18 embeddings from a pretrained network capture semantic content that no handmade feature can represent directly.

I also implemented 3 extensions: a GLCM co-occurrence descriptor, a Gabor filter bank, and custom object detectors targeting bananas and blue recycling bins. The system is implemented in C++ with OpenCV and supports both a live query mode and offline batch feature extraction.

Table 1 summarises every descriptor implemented in this project, its vector length, and the distance metric used at query time.

Table 1: Feature vector sizes and distance metrics for all methods.

Method	Key parameter(s)	Vector size	Distance
<code>baseline</code>	7×7 center patch, 3 ch.	147	SSD
<code>rg_hist</code>	2-D RG chromaticity, 16 bins	256	Histogram intersection
<code>rgb_hist</code>	3-D RGB, 8 bins	512	Histogram intersection
<code>multi_hist</code>	Top+bottom RGB, 8 bins each	1024	Weighted intersection
<code>texture_color</code>	RGB (8 bins) + Sobel mag. (16 bins)	528	Weighted intersection
<code>cooccurrence</code>	GLCM Haralick, 5 statistics	5	SSD
<code>banana</code>	Yellow HSV blob, 4-value descriptor	4	Weighted SSD
<code>trash_can</code>	Blue HSV blob, 4-value descriptor	4	Weighted SSD
<code>gabor</code>	4 orient. × 2 scales × 8 bins	64	Avg. hist. intersection
<code>dnn</code>	ResNet18 pool layer	512	Cosine distance

1. Baseline: 7×7 Center-Patch SSD

The 7×7 center crop of the image is flattened into a raw BGR pixel vector (147 values). Two images are compared with sum of squared differences:

$$\text{SSD}(\mathbf{a}, \mathbf{b}) = \sum_i (a_i - b_i)^2$$

The SSD code is written from scratch without OpenCV functions; an image compared with itself gives exactly 0. This method is sensitive to framing and scale but is fast and serves as a baseline for all subsequent methods.

Query: pic.1016.jpg **Matches:** 0986, 0641, 0547, 1013



Figure 1: Baseline SSD results for pic.1016.jpg. Matches share similar center region pixel values.

2. Histogram Matching

2.1. RG Chromaticity Histogram (rg_hist)

Color is separated from brightness by normalizing: $r = R/(R+G+B)$, $g = G/(R+G+B)$. Pure black pixels are skipped to avoid division by zero. The (r, g) pairs are binned into a $16 \times 16 = 256$ -bin normalized histogram. Illumination intensity changes do not affect this representation.

Distance is histogram intersection, implemented from scratch:

$$d(\mathbf{a}, \mathbf{b}) = 1 - \sum_i \min(a_i, b_i)$$

Query: pic.0164.jpg **Matches:** 0080, 1032, 0461

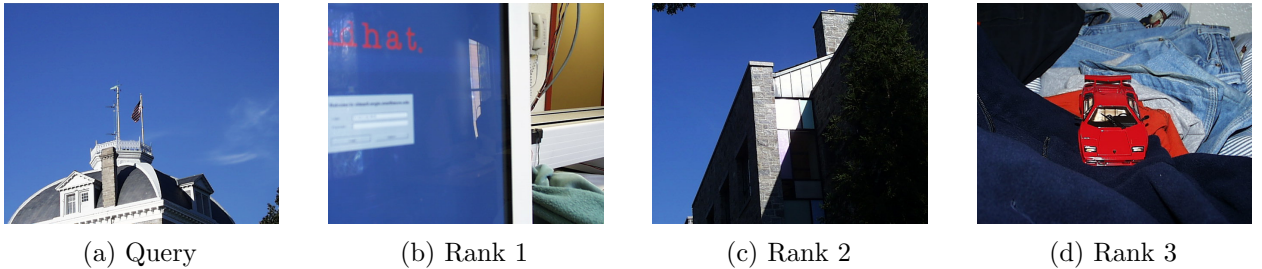


Figure 2: RG chromaticity histogram results for pic.0164.jpg. The illumination invariant representation groups images by hue distribution regardless of exposure.

2.2. 3D RGB Histogram (rgb_hist)

Each BGR channel is independently quantized into 8 bins ($8^3 = 512$ bins total). The histogram is normalized by pixel count. Unlike RG chromaticity, brightness is retained, which helps separate dark from bright scenes sharing a similar hue.

Query: pic.0164.jpg **Matches:** 0110, 1032, 0092

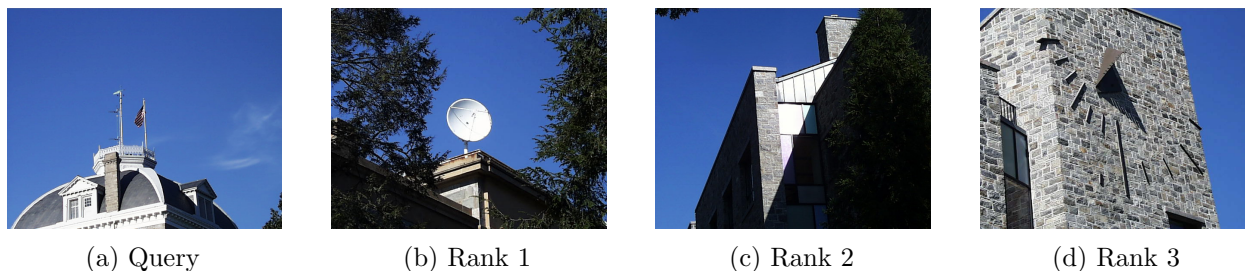


Figure 3: 3D RGB histogram results for `pic.0164.jpg`. `pic.1032.jpg` appears in both histogram methods, suggesting it is a genuinely close color match.

3. Spatial Histogram: Top/Bottom Split (multi_hist)

A global histogram loses spatial structure. Splitting at the horizontal midpoint and computing independent 8^3 -bin RGB histograms for each half (1024 values total) allows the distance metric to distinguish images that share a global palette but differ in where those colors appear. The two half distances are averaged with equal weight.

Query: pic.0274.jpg **Matches:** 0273, 1031, 0409

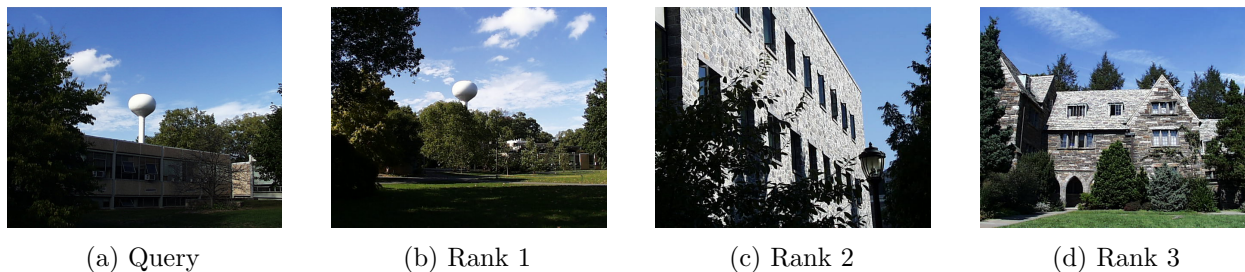


Figure 4: Top/bottom spatial histogram results for `pic.0274.jpg`. The spatial split recovers matches that share the same color layout (e.g., sky above, landscape below).

4. Texture + Color (texture_color)

A global 3D RGB histogram (512 values) is concatenated with a Sobel gradient magnitude histogram (16 values). The texture component is built by converting to grayscale, computing $M = \sqrt{S_x^2 + S_y^2}$ at every pixel, capping at 1000 to avoid distorting low magnitude bins, and normalizing. Smooth regions (sky, water) pile into the low bins; textured regions (foliage, crowds) spread energy toward higher bins. Total feature: 528 values. Distance is a 50/50 weighted average of the color and texture intersection values.

Query: pic.0535.jpg **Matches:** 0853, 0605, 0708, 0629

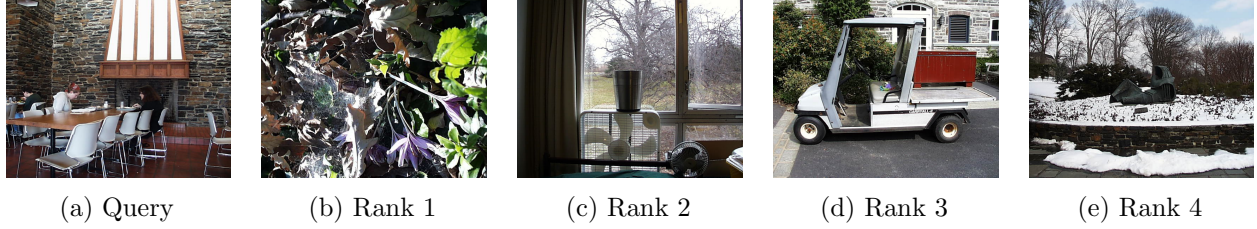


Figure 5: Texture + color results for `pic.0535.jpg`.

Comparison with Tasks 2 and 3: For the same query, `rg_hist` returns 0733, 0628, 0658 and `rgb_hist` returns 0285, 0628, 0952, results driven purely by color distribution. The `texture_color` matches (0853, 0605, 0708) are almost entirely different, confirming that the texture term is contributing meaningfully. Images that share the query’s edge density profile but differ in color will rank higher under this method than under any histogram only approach.

5. Deep Network Embeddings (dnn)

Pre computed ResNet18 embeddings (512 dimensional vectors from the global average pooling layer) are loaded from CSV. These features encode semantic content learned from 1M ImageNet images. Distance is cosine distance:

$$d(\mathbf{a}, \mathbf{b}) = 1 - \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Cosine distance compares vector orientation rather than magnitude, which is more appropriate for high dimensional vectors where direction carries the semantic information.

Query: `pic.0893.jpg` **Matches:** 0897, 0136, 0146

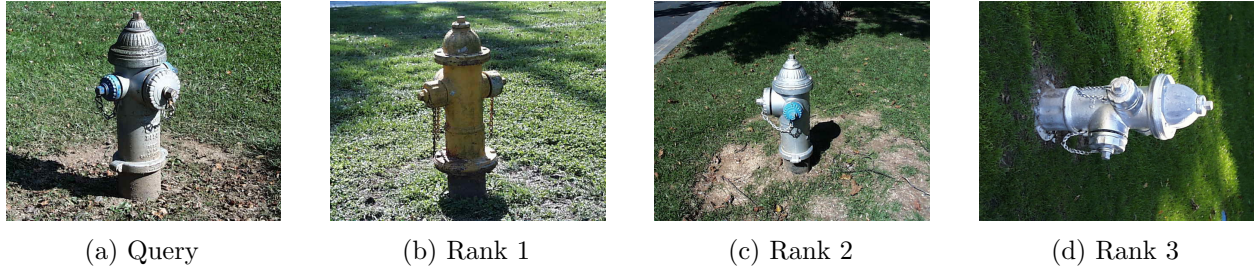


Figure 6: DNN results for `pic.0893.jpg`.

Query: `pic.0164.jpg` **Matches:** 1032, 0213, 0690

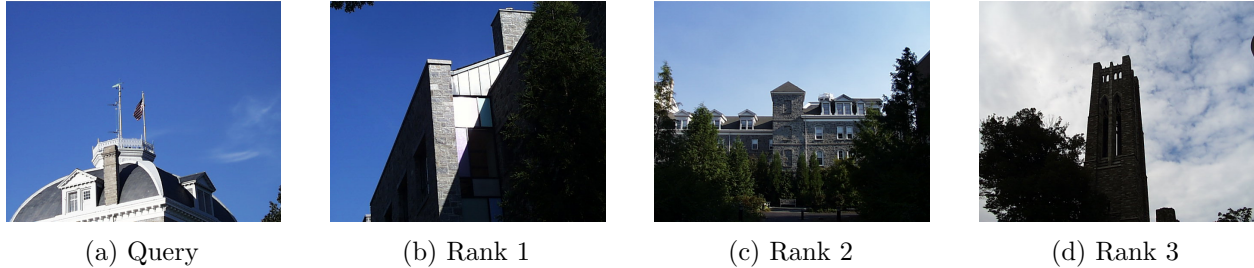


Figure 7: DNN results for `pic.0164.jpg`. `pic.1032.jpg` appeared in both `rg_hist` and `rgb_hist` results for the same query. When both DNN and classic features agree on a match, it is a strong signal of genuine visual similarity.

6. DNN vs. Classic Feature Comparison

Three query images (0734, 0948, and 1072) were run against DNN, `rgb_hist`, and `texture_color`. Results are shown in Table 2.

Table 2: Top-5 results: DNN cosine distance vs. handmade features.

Rank	DNN (cosine)	<code>rgb_hist</code>	<code>texture_color</code>
<i>Query: pic.0734</i>			
1	pic.0731	pic.0450	pic.0191
2	pic.0735	pic.0577	pic.0255
3	pic.0739	pic.0140	pic.1033
4	pic.0743	pic.1088	pic.0814
5	pic.0745	pic.0076	pic.0450
<i>Query: pic.0948</i>			
1	pic.0930	pic.0217	pic.0891
2	pic.0960	pic.0675	pic.0661
3	pic.0928	pic.0735	pic.0675
4	pic.0972	pic.1069	pic.0681
5	pic.0962	pic.0582	pic.0542
<i>Query: pic.1072</i>			
1	pic.0143	pic.0937	pic.0701
2	pic.0863	pic.0732	pic.0563
3	pic.0329	pic.0673	pic.0234
4	pic.0144	pic.1069	pic.0964
5	pic.0257	pic.0701	pic.0974

For `pic.0734` and `pic.0948`, DNN returns sequentially numbered images: 0731–0745 and 0928–0972 respectively. These are frames from the same photo shoot. The model recovers the temporal structure of the original shoot with no explicit knowledge that these images are related. No handmade feature could do this, because the relationship is semantic rather than statistical. Classic features return images from entirely different parts of the database that happen to share the query’s color histogram or edge density.

Classic features are not without value. They outperform DNN when the goal is matching by palette regardless of content, when the database domain differs significantly from ImageNet, or when texture or material properties are the intended similarity criterion rather than object identity.

Extensions

Three extensions are implemented: a GLCM co-occurrence descriptor, a Gabor filter bank texture feature, and custom HSV-based object detectors for bananas and blue recycling bins.

7. Extension 1: GLCM Co-occurrence Texture (cooccurrence)

Description

The Gray Level Co-occurrence Matrix (GLCM) captures spatial texture regularity by recording how often pairs of gray level values (i, j) appear at a given spatial offset [2]. Four offset directions are used: horizontal $(1, 0)$, vertical $(0, 1)$, and two diagonals, and five Haralick statistics are extracted per direction and averaged for rotation invariance: energy, entropy, contrast, homogeneity, and correlation. The image is quantized to 8 gray levels; all statistics are normalized to $[0, 1]$. Feature vector: 5 values. Distance: SSD.

This captures regularity that gradient histograms cannot; for example, two images with identical Sobel histograms can have very different GLCM statistics if one has periodic structure and the other does not.

Query: pic.0274.jpg **Matches:** 0297, 0070, 0537, 0348

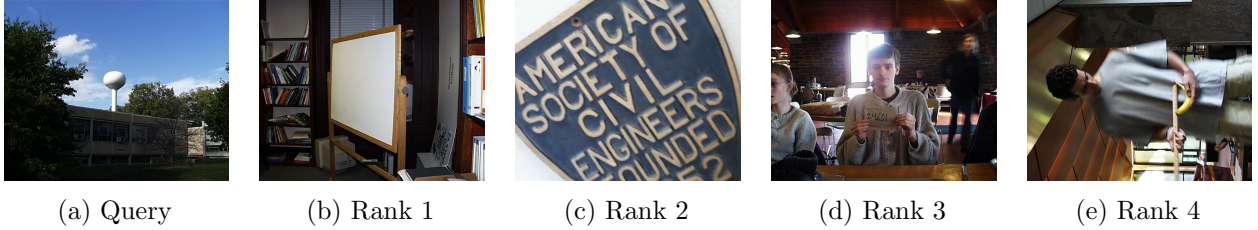


Figure 8: GLCM co-occurrence results for `pic.0274.jpg`. Images are grouped by spatial texture regularity independent of color.

8. Extension 2: Gabor Texture Feature (gabor)

Description

Implementation: 4 orientations $\times$ 2 wavelengths ($\lambda = 4$ px and 8 px), $\sigma = \lambda/\pi$, 15×15 kernel. Each filter response magnitude is binned into a normalized 8-bin histogram and concatenated: $4 \times 2 \times 8 = 64$ values total. Distance is the average histogram intersection across all per filter histograms, keeping the result in $[0, 1]$ regardless of how many filters are stacked.

Query: pic.0535.jpg **Matches:** 0154, 0355, 0393, 0941, 0975



Figure 9: Gabor texture results for `pic.0535.jpg`.

9. Extension 3: Custom Object Detectors: Banana and Trash Can

Description

Two domain specific detectors find objects with distinctive color signatures: yellow bananas and blue plastic recycling bins. Both detectors use the same 4-value structure but target different HSV ranges.

Target pixels are isolated with `cv::inRange` in HSV space:

Table 3: HSV ranges used by the two detectors (OpenCV scale: H in $[0,179]$).

	Banana	Trash Can
Hue	15–40	100–130
Saturation	80–255	80–255
Value	80–255	40–255

Feature vector (4 values, same for both detectors):

1. **Fraction**: proportion of pixels in the target color range
2. **var_x**: spatial variance of matching pixels along x (coordinates normalized to $[0,1]$)
3. **var_y**: same along y
4. **Coherence** = $1/(1 + \text{var}_x + \text{var}_y)$, high for a tight blob, low for diffuse color

Coherence is the key discriminator. Blue sky fills the frame uniformly, giving spatial variance $\approx 1/12$ per axis (the uniform distribution variance on $[0,1]$) and coherence ≈ 0.86 . A blue trash can is a compact object with coherence > 0.97 . Distance: weighted SSD with weights $[4, 1, 1, 2]$, emphasizing fraction.

For the banana detector, pure HSV thresholding ranked sunsets above actual banana images. The fix is the combined `banana_dnn` method, which averages DNN cosine distance with banana weighted SSD at equal weight:

$$d_{\text{combined}} = 0.5 d_{\text{cosine}} + 0.5 d_{\text{banana}}$$

DNN provides semantic grounding; the blob feature adds specificity to yellow toned food objects.

Banana Results

Known banana images in the database: 0342, 0343, 0344, 0345, 0347, 0833. The feature weights are $[1, 1, 1, 4]$; coherence is emphasised over fraction because the banana images vary widely in framing (fraction ranges from 0.015 to 0.142 across the 6 known images) while coherence is consistently high (0.89–0.99) across all of them.

Query: pic.0343.jpg **Top 5 matches:** 0345, 1010, 0496, 0346, 0818

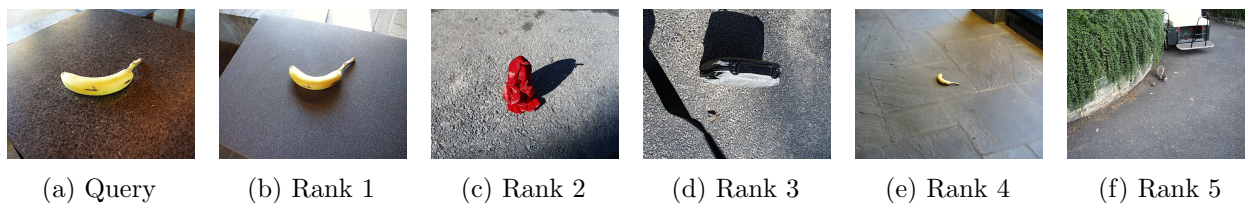


Figure 10: `banana_dnn` top-5 for `pic.0343.jpg`. Rank 1 (`pic.0345`) is a confirmed banana image.

Query: pic.0347.jpg **Top 5 matches:** 0714, 0474, 0151, 0028, 0320

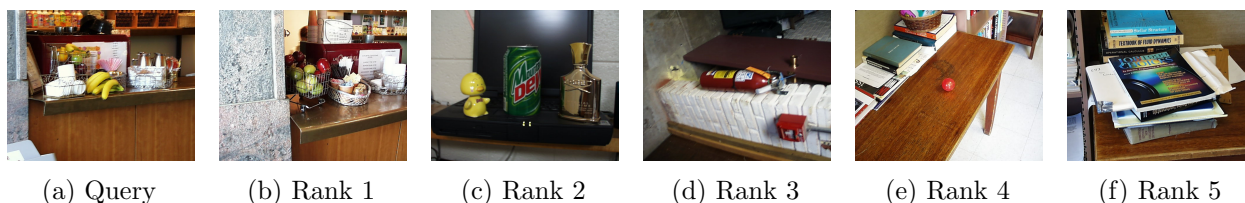


Figure 11: `banana_dnn` top-5 for `pic.0347.jpg`.

Trash Can Results

The two confirmed blue recycling bins in the database are 0287 and 0291. The combined `trash_can_dnn` method (0.7 DNN + 0.3 blue blob) is used here; DNN carries higher weight because the two known bin images differ significantly in blue fraction (0.249 vs 0.172) making the blob term alone insufficient.

Query: pic.0287.jpg **Top 5 matches:** 0291, 0289, 0288, 0591, 0201

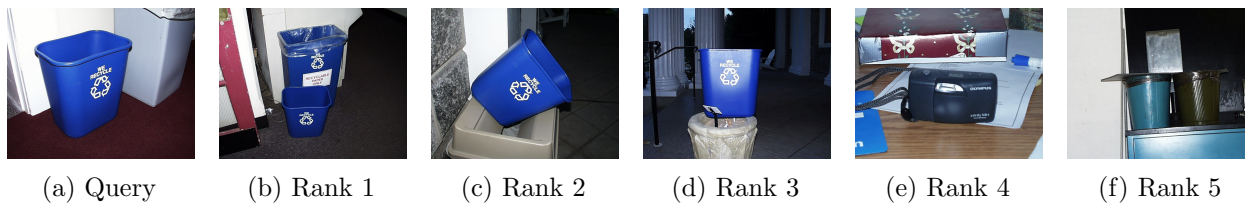


Figure 12: `trash_can_dnn` top-5 for `pic.0287.jpg`. Rank 1 (`pic.0291`) is the other confirmed blue recycling bin.

Reflection

Going into this project I thought the hardest part would be the histograms and the distance metrics. Those turned out to be mechanical. The part that actually surprised me was how clearly the DNN results exposed the semantic gap. When I ran pic.0734 and saw the top five matches all had sequential numbers (0731, 0735, 0739, 0743, 0745), and realized the model was effectively recovering the order images were taken in a camera roll. That was not something I anticipated from reading about embeddings abstractly.

The banana task was a useful reminder that single cue features fail in predictable ways. Yellow thresholding found sunsets before it found bananas. The fix was combining DNN distance with the spatial blob feature. It worked specifically because the two failure modes are independent. The DNN doesn't care about exactly how yellow something is and the blob feature doesn't know what a banana is. Together they compensate for each other's blind spots.

Working through the texture features also sharpened my understanding of what histograms actually discard. A histogram tells you how much of something is present but nothing about where or in what arrangement. Every method from the spatial split onward is really just trying to recover some of that spatial information in different ways. This makes the DNN result even more striking, since it gets spatial and semantic information "for free".

Acknowledgements

The olympus image database and ResNet18 embedding CSV were provided by the course instructor. Chapter 8 of Shapiro and Stockman [1] was consulted for background on histogram methods and co-occurrence matrices. The OpenCV documentation [5] was used for `cv::getGaborKernel`, `cv::filter2D`, and `cv::inRange`. The Haralick et al. [2] paper was used as the primary reference for GLCM feature definitions. Claude (Anthropic) was used as an AI assistant for planning code structure, explaining concepts, troubleshooting implementation issues, drafting concise and descriptive code comments, templating this report, and generating the project README from provided build commands and relevant image filenames in the olympus photo directory.

References

- [1] L. G. Shapiro and G. C. Stockman, *Computer Vision*. Prentice Hall, 2001.
- [2] R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973.
- [3] A. K. Jain and F. Farrokhnia, "Unsupervised texture segmentation using Gabor filters," *Pattern Recognition*, vol. 24, no. 12, pp. 1167–1186, 1991.
- [4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, pp. 770–778, 2016.
- [5] OpenCV Development Team, *OpenCV 4 Documentation*, 2024. [Online]. Available: <https://docs.opencv.org/4.x/>